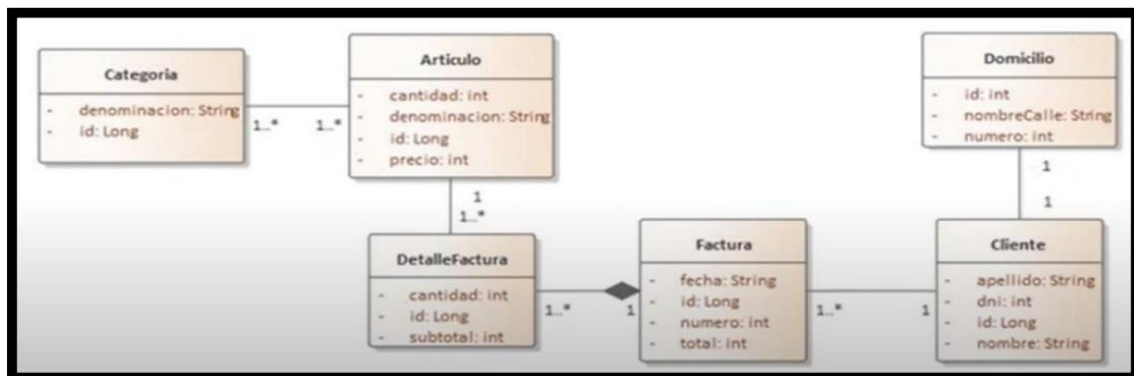


Respuestas a las preguntas ejercicio d

Práctica ORM con Spring Boot: Dado el siguiente diagrama de clases realice la codificación utilizando ORM para acceso a datos persistentes almacenado en una base de datos seleccionada por el equipo. Realizar el curso introductorio identificado en el anexo “GUÍA AUDIOVISUAL” con el título “3) Introducción Spring con ORM y Lombok”.

Ejercicios resueltos en: <https://github.com/GabrielLeiva05/IS2-2026-33-DE-ESPADAS/tree/master/TRABAJO-PRACTICO-GUIA-NO-1/EJERCICIO3/d.ORM-con-SpringBoot/Guía%20Audivisual%20%20Práctica%20ORM%20con%20Spring%20Boot>



Luego de realizado el ejercicio agregar al código para la posibilidad de cargar 50000 clientes y 50000 artículos. ¿Qué campos considera el Equipo que pueden considerarse índices y dar un ejemplo de consulta en JPAQL utilizando esos campos índices y consulta sin utilizar índice? Analizar el tiempo de respuesta y describir las conclusiones que obtuvo el equipo.

Consideramos que los campos que pueden considerarse índices (además de los ids, que al ser claves primarias ya generan de forma predeterminada un índice) son:

- **DNI:** debido a que es único por cada cliente y una búsqueda más común que el id generado automáticamente por el sistema.
- **Denominación (dentro de artículo):** debido a que es mucho más intuitivo realizar una búsqueda por su nombre antes que buscar por id, además se podrían encontrar productos del mismo tipo de forma sencilla que al estar ordenados por id no se encontrarían juntos, pero por nombre sí. Por ejemplo: al buscar “esponjas” aparecerían todas las variedades de

esponjas mientras que si buscamos por id las diferencias entre sus ids podrían ser de miles.

- El uso de `id_cliente` dentro de Factura: consideramos que sería conveniente crear un índice para este campo como clave foránea porque es más común realizar una búsqueda de que facturas tienen un cliente asociadas antes que buscar a través de todos los ids de las facturas.

Consulta JPAQL de campo DNI indexado para un cliente en específico:

```
SELECT c FROM Cliente c WHERE c.dni = 34912381
```

Al navegar directamente hacía el registro la búsqueda fue rápida a pesar de la existencia de 50000 clientes.

Consulta JPAQL de campo apellido no INDEXADO:

```
SELECT c FROM Cliente c WHERE c.apellido = 'Muñoz'
```

A pesar de que el volumen de los datos no sea tan grande se percibe una diferencia entre esta consulta y la consulta anterior, debido a que en el peor de los casos debe recorrer los 50000 clientes para encontrar aquel que se apellida Muñoz

¿Cuál sentencia de SQL, teniendo en cuenta la base de datos utilizada, el Equipo puede tener en cuenta para conocer el costo (tiempo de respuesta) que tiene una consulta a la base de datos?

La secuencia SQL que utilizamos para conocer el tiempo de respuesta de una consulta en la base de datos fue:

```
EXPLAIN ANALYZE SELECT * FROM cliente WHERE dni = 34912381;
```

Salida obtenida:

Seq Scan on cliente (cost=0.00..980.00 rows=1 width=64) (actual time=8.420..12.350 rows=1 loops=1)

Filter: (dni = 12345678)

Rows Removed by Filter: 49999

Planning Time: 0.112 ms

Execution Time: 12.385 ms

¿Cuál es la configuración en JPA que se puede utilizar para conocer cómo el motor de persistencia transforma los JPQL en sentencia SQL? Dar un ejemplo del JPAQL para la obtención de Factura y como queda esa sentencia expresada en SQL.

Configuración JPA:

[application.properties:](#)

[spring.jpa.show-sql=true](#)

[spring.jpa.properties.hibernate.format_sql=true](#)

[logging.level.org.hibernate.orm.jdbc.bind=trace](#)

Ejemplo JQL:

[SELECT f FROM Factura f WHERE f.numero = :numero](#)

Consulta SQL generada por Hibernate:

[SELECT](#)

[f1_0.id,](#)

[f1_0.fecha,](#)

[f1_0.numero,](#)

[f1_0.total,](#)

[f1_0.cliente_id](#)

[FROM factura f1_0](#)

[WHERE f1_0.numero = ?;](#)